

Úvod do databázových systémů

Kapitola 1
Základy softwarového inženýrství

Proč softwarové inženýrství

- v začátcích počítačů byly programy tvořeny malými týmy či jednotlivci
 - úzce specializované programy, obvykle vědecké a technické výpočty
 - programovány v jazyce s malou úrovní abstrakce (strojový kód, assembler, nanejvýš Fortran)
- s příchodem integrovaných obvodů se možnosti počítačů zvětšily (rychlejší, větší paměť) – potenciál pro rozvoj rozsáhlejších SW produktů (bankovních IS atp.)

Proč softwarové inženýrství

- aplikace tradičních programovacích metod (víceméně ad hoc návrh a implementace) na větší systémy vedla k problémům
 - velké zpoždění při vývoji
 - mnohonásobné zvýšení ceny oproti původním odhadům
 - chybovost nového SW
- tzv. softwarová krise

Proč softwarové inženýrství

- rozsáhlé projekty jsou zcela odlišné od malých
 - nelze přímo přenést zkušenosti
 - mnoho práce zabere
 - naplánovat, jak rozdělit projekt do dílčích celků (modulů)
 - specifikace činnosti a rozhraní modulů
 - naplánovat interakce modulů
 - až potom přichází vlastní „kódování“
 - napsání a odladění samostatných modulů
 - sestavení modulů do výsledého celku
 - testování celku (moduly obvykle hned nespolupracují správně, ačkoli samostatně jsou v pořádku)

Proč softwarové inženýrství

- odhad náročnosti prací
 - 1/3 celkové práce je plánování
 - 1/6 kódování
 - 1/4 testování modulů
 - 1/4 testování systému
- kódování je vlastně to nejjednodušší, nejvíc práce dá správně vše naplánovat
 - rozdělit projekt na moduly
 - navrhnout a zajistit bezchybnou interakci modulů

Co je software (SW)

- často se míní, že software je pouze program, či kód
- software je nutno chápat komplexněji
 - programy
 - dokumentace
 - systémová
 - uživatelská
 - konfigurační data
- 2 typy produktů
 - generický (vyvíjen na základě potřeb trhu, koupí si ho, kdo chce – textové editory, OS, apod)
 - na zakázku

Co je softwarové inženýrství

- SW inženýrství

- aplikace inženýrských metod na vývoj SW

- specifikace

- vývoj

- testování

- údržba

- management

- definice

- aplikace disciplinovaného, systematického, měřitelného přístupu na vývoj a údržbu SW

Co je softwarové inženýrství

- SW inženýrství
 - vyžaduje dělbu práce a spolupráci mnoha profesí
 - vyžaduje organizaci (řízení)

Co je softwarové inženýrství

- SW engineering process
 - množina provázaných aktivit, které převádějí nějaké vstupy na nějaké výstupy, přičemž spotřebovávají nějaké zdroje
 - základní oblasti
 - vývoj SW
 - správa a údržba SW
 - obsluha SW
 - aktivity mohou být typicky
 - shromažďování požadavků na systém (requirements)
 - návrh (design)
 - implementace (construction)
 - testování (testing)
 - správa konfigurace (configuration management),

Co je softwarové inženýrství

- SW process management

- SW procesy je nutno řídit

- je třeba zajistit efektivitu, účelnost a systematičnost
 - směřování k cíli,
 - nízká chybovost,
 - malá nutnost přepracovávání,
 - nedělat totéž víckrát, ...

- řízení procesů

- obvykle na úrovni jednotlivých projektů
 - může být ale přínosné zavést procesní infrastrukturu v rámci celé organizace
 - definice standardních procesů
 - pravidla aplikace jednotlivých procesů
 - popis, jak jednotlivé procesy uskutečnit (implementovat)

Co je softwarové inženýrství

- SW process management
 - řízení SW procesů přímo ovlivňuje řadu klíčových vlastností výsledného produktu
 - kvalitu
 - cenu
 - dostupnost
 - trvání vývoje
- při tvorbě SW produktu se uplatňuje řada SW procesů
 - jejich výběr, uspořádání a vzájemné vztahy jsou popsány nějakým modelem **životního cyklu SW produktu**

Životní cyklus

- model životního cyklu SW produktu
 - modely mohou být různé
 - model je vždy nějaká idealizace, nelze jej aplikovat dogmaticky – adaptace na konkrétní situaci
 - model životního cyklu definuje
 - klíčové procesy jako takové
 - časové uspořádání procesů
 - logické vztahy a závislosti procesů
 - výstup jednoho procesu často poskytuje vstup jinému procesu
 - kontrolní mechanismy, kriteria startu a konce procesů
 - např. revize projektu, schválení zákazníkem, výsledek testu, usnesení realizačního týmu, apod...

Životní cyklus

- model životního cyklu SW produktu
 - v principu existuje několik základních typů
 - lineární
 - jednotlivé procesy (fáze tvorby díla) na sebe sekvenčně navazují, k předchozím procesům se obvykle nevracíme
 - iterativní
 - produkt je vyvíjen v definovaných cyklech, v každém cyklu dojde k nějakému přírůstku
 - agilní
 - vychází z iterativní myšlenky, je vysoce adaptivní, tj. dovede pružně reagovat na změny, ovšem dopředu nelze napláňovat vůbec nic
 - vyžaduje velké zapojení zákazníka

Životní cyklus

- základní procesy (fáze) – hrubě
 1. analýza – co bude systém umět
 - sběr požadavků
 - modelování procesů, modelování dat
 2. návrh – jak to bude systém dělat
 - volba technologie, návrh architektury
 - volba frameworků
 - volba způsobu ukládání dat
 - rozdělení úkolů, způsob komunikace
 3. realizace – tvorba systému
 - programování (zdrojové kódy...)
 - dokumentování (systémové, uživatelské)
 - sestavení (instalační balíčky...)

Životní cyklus

- základní procesy (fáze) – jemněji
 1. předběžná analýza (specifikace hlavních cílů)
 2. analýza (specifikace požadavků, requirements)
 3. návrh (projektová studie, design)
 4. implementace (construction)
 5. testování (testing)
 6. zavádění systému
 7. zkušební provoz
 8. rutinní provoz a údržba
 9. reengineering

Životní cyklus

1. předběžná analýza, specifikace cílů
 - shromáždění požadavků uživatelů, cílů organizace
 - odhad doby realizace a nákladů
 - cílem je pouze sestavit základní rámec požadavků, cílů a funkcí, ne je podrobněji rozebírat

Životní cyklus

2. analýza systému, specifikace požadavků
 - rozbor předběžné analýzy
 - klíčová důležitost
 - veškeré chyby ve struktuře dat i systému, které se zde neodhalí, mohou být později velice obtížně odstranitelné

Životní cyklus

3. návrh systému

- výsledek analýzy systému
 - definuje architekturu, komponenty, rozhraní, apod.
- projektová studie
 - obsah smlouvy o návrhu a realizaci IS
 - časový harmonogram
 - cena vyvíjeného projektu
 - konkrétní implementace systému (logický datový model, fyzický datový model...)
 - podmínky zavádění do praxe (HW, SW, atd.)
 - podmínky celkového předání IS
 - záruční servis

Životní cyklus

4. implementace

- vlastní programování
 - programátoři
 - zodpovědný analytik
- definice vstupů a výstupů
- naprogramování funkcí
- vyladění interakce funkcí
- ověření funkcí
- příprava co možná nejreprezentativnějších testovacích dat

Životní cyklus

5. testování

- připravené testy na (více či méně) hotovém systému
- nutnost vyzkoušet reakce systému na všechny možné vstupy, oprava chyb
- často se provádí mimo reálné prostředí (letectví, zdravotnictví atd.)

Životní cyklus

6. zavádění systému

- instalace, uvedení do provozu
- předání dokumentace
- školení uživatelů
 - nejprve vedoucí a pak podřízené
- nesmí se podcenit

Životní cyklus

7. zkušební provoz

- povinnost okamžitého servisu
- odstraňování chyb
- dořešení dodatečných požadavků

Životní cyklus

8. rutinní provoz a údržba

- závěrečná fáze projektu, provoz a používání systému
- zajištění optimálního provozu
- zabezpečení systému
- záloha dat
- ...

Životní cyklus

9. reengineering

- přehodnocení požadavků na systém
- pokud nelze nové požadavky uspokojit úpravou stávajícího systému, jdeme zase na začátek životního cyklu

Životní cyklus

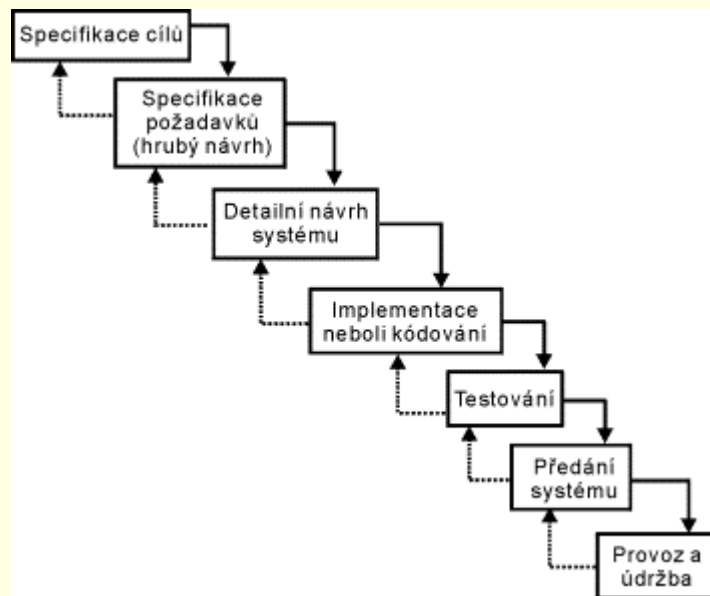
- platí, že většina chyb systému je způsobena špatným návrhem, nikoli špatnou implementací
 - programátoři naprogramují přesně to, co se jim řeklo, i když je to špatně
 - čím později (ve vyšší fázi vývoje) se přijde na chybu, tím dražší bývá její náprava

Modely životního cyklu

- uvedeme si několik ukázek modelů životního cyklu
- model vodopád (waterfall)
 - ryze lineární model, zcela prediktivní – vše je přesně naplánováno
 - při návrhu se provádějí postupně jednotlivé etapy životního cyklu, které na sebe navazují a neprolínají se
 - etapy se provádějí podle stanoveného plánu a nevracíme se k nim
 - dokončená etapa je vstupem etapy následující

Modely životního cyklu

- model vodopád



Modely životního cyklu

- model vodopád

- výhody

- rychlý a levný, pokud se nevyskytnou problémy
 - pevná struktura vývoje SW, šetření zdrojů

- nevýhody

- skutečné projekty se v pevně definovaných krocích obvykle nedají řešit
 - výsledek máme až po skončení poslední fáze vývoje, tj. chyba na začátku vede k nutnosti přepracovat vše
 - nesmírně drahé
 - extrémní nároky na správnou analýzu

- i když má nevýhody, je to pořád lepší než ad hoc (náhodný) vývoj bez řízení

Modely životního cyklu

- prototypový model

- předpoklad změn v průběhu vývoje

- iterativní, částečně adaptivní

- prototyp

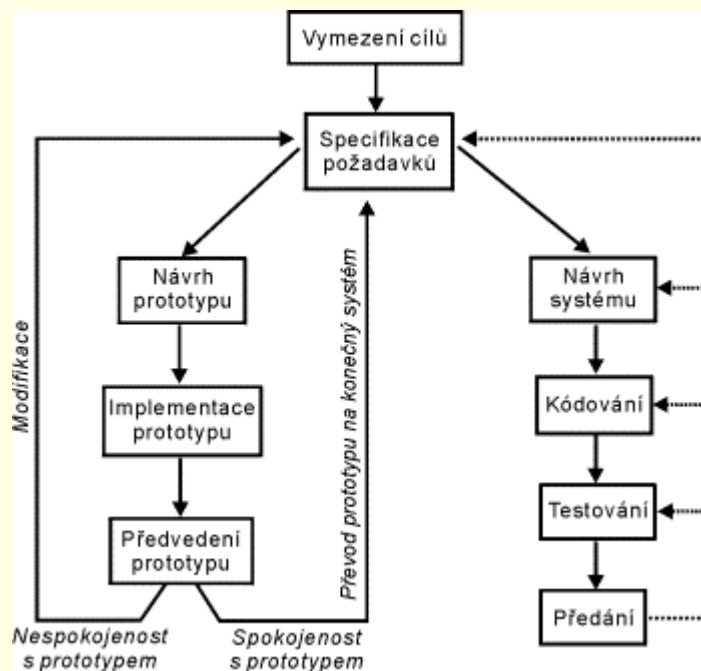
- seznámení zákazníků s prvními verzemi systému co nejdříve
 - zjednodušená implementace celku nebo plná implementace části systému
 - prezentace všech GUI, základní funkčnost

- iterativní upřesňování požadavků a jejich implementace do prototypu, dokud zákazník není spokojen

- nakonec implementace

Modely životního cyklu

■ prototypový model



Modely životního cyklu

- prototypový model

- výhody

- umožňuje přesně definovat požadavky uživatelů
 - reakce na změny zadání či požadavků

- nevýhody

- u větších systémů poměrně náročné
 - vlivem iterací se může „rozpliznout“ struktura systému
 - obtížné řízení
 - nevíme, v jaké fázi vývoje se nacházíme
 - nutno omezit, kolik prototypů budeme vytvářet
 - přesně stanovit termíny předání prototypů

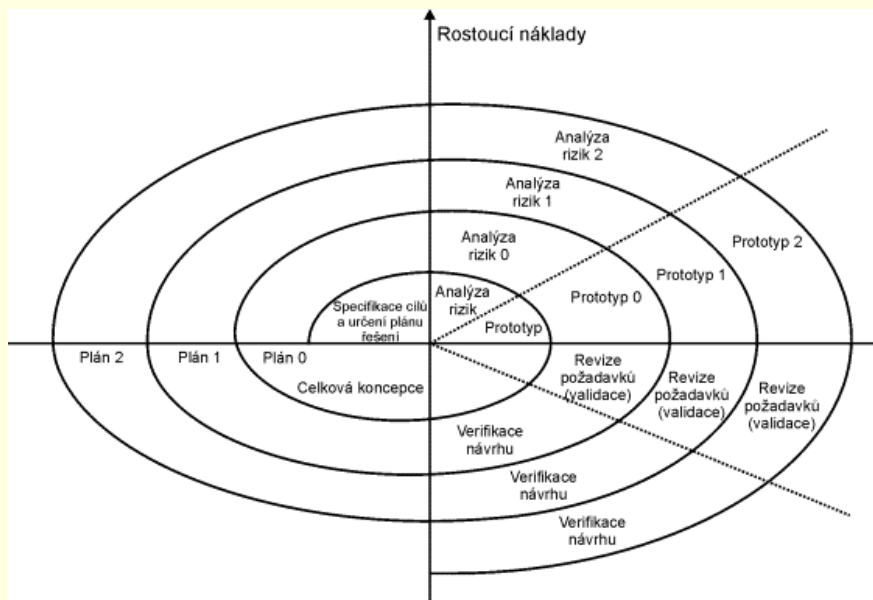
Modely životního cyklu

■ model spirála

- kombinace prototypového modelu, průběžné analýzy rizik a průběžné implementace
 - iterativní, částečně adaptivní
- opakování vývojových fází tak, že v každém dalším kroku se na již ověřenou část nabalí další části
- postup vývoje je podobný jako ve vodopádu
 - specifikace cílů a určení plánu řešení
 - vyhodnocení alternativ řešení a analýza rizik
 - vývoj prototypu dané úrovně, předvedení, vyhodnocení
 - revize požadavků - validace (pracuje prototyp jak má?)
 - verifikace – ověření: je celkový výstup daného kroku v souladu se zjištěnými požadavky?

Modely životního cyklu

■ model spirála



Modely životního cyklu

■ model spirála

■ výhody

- využívá ověřené kroky vývoje a analýzou rizik předchází chybám
- umožňuje konzultovat požadavky zákazníků v jednotlivých krocích a modifikovat systém podle upřesněných požadavků
- prvotní verze systému lze sledovat během vzniku

■ nevýhody

- vyžaduje neustálou spolupráci zákazníků
- neumožňuje přesné naplánování termínů, cen a jednotlivých výstupů
- závislost na správně provedené analýze rizik

Informační systém (IS)

■ IS

- typický představitel SW produktu
- systém vzájemně propojených informací a procesů, které s informacemi pracují
- procesy
 - funkce, transformující vstupy na výstupy

■ úloha IS

- jeden z hlavních faktorů efektivního řízení a konkurenceschopnosti podniku
- závislost podniku na kvalitních a včasných informacích

Databáze v IS

- databáze je důležitou součástí IS
 - musí umožňovat
 - uchovávání dat
 - manipulaci s daty (vkládání, aktualizaci, mazání)
 - efektivní vyhledávání dat
 - propojování dat, komplexní informace
 - chybný návrh může způsobovat chybné dotazy nebo i nemožnost dosáhnout určitých informací
- ve strukturované analýze obvykle návrh databáze spadá do datové analýzy